QUESTION 13

Aim

To create document datasets of increasing size, build FAISS indexes, measure indexing time and average search latency, plot the measurements, and analyze the scalability of exact vector search.

Objectives

- Generate approximately 100, 500, 1,000, 5,000 and 10,000 documents.
- Convert documents into numerical vectors.
- Build a FAISS index for each dataset size.
- Measure indexing time and average search latency.
- Generate scalability graphs.
- Interpret the observed trend.

Methodology

Synthetic documents are generated from NLP and information-retrieval topics. TF-IDF converts the documents into numerical vectors. The vectors are normalized and stored in FAISS IndexFlatIP. Because normalized inner product corresponds to cosine similarity, the index performs exact similarity search.

Algorithm

1. Generate the selected number of documents.

2. Create TF-IDF vectors.

3. Convert vectors to float32 and normalize them.

4. Create IndexFlatIP and add the vectors.

5. Measure index construction time.

6. Select query vectors and perform top-5 searches.

7. Calculate average query latency.

8. Repeat for all five dataset sizes.

9. Save measurements and plot the results.

Implementation and Measurement

Formulas

Indexing Time = $t_2 - t_1$

Search Latency$_i$ = $t_{2i} - t_{1i}$

Average Search Latency = Σ Search Latency$_i$ / N

N is the number of executed search queries.

Python Program – IDLE Compatible

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import faiss
from sklearn.feature_extraction.text import TfidfVectorizer

SIZES = [100, 500, 1000, 5000, 10000]
TOP_K = 5
N_QUERIES = 50

topics = [
    "machine learning algorithms improve prediction accuracy",
    "natural language processing analyzes human language",
    "data science uses statistics and computational methods",
    "database systems store and retrieve structured information",
    "cloud computing provides scalable computing resources",
    "information retrieval finds relevant documents for queries",
    "artificial intelligence supports intelligent applications",
    "software engineering uses systematic development practices"
]

def make_documents(n):
    docs = []
    for i in range(n):
        topic = topics[i % len(topics)]
        docs.append("Document " + str(i) + ". " + topic +
                    ". This document discusses indexing search retrieval "
                    "embeddings and scalable information processing.")
    return docs

def benchmark(docs):
    vectorizer = TfidfVectorizer(max_features=512)
    vectors = vectorizer.fit_transform(docs).toarray().astype("float32")
    faiss.normalize_L2(vectors)
    index = faiss.IndexFlatIP(vectors.shape[1])

    start = time.perf_counter()
    index.add(vectors)
    indexing_time = time.perf_counter() - start

    rng = np.random.default_rng(42)
    q_count = min(N_QUERIES, len(vectors))
    query_ids = rng.choice(len(vectors), q_count, replace=False)
    latencies = []

    for qid in query_ids:
        q = vectors[qid].reshape(1, -1)
        start = time.perf_counter()
```

```
        index.search(q, TOP_K)
        latencies.append((time.perf_counter() - start) * 1000)

    return indexing_time, np.mean(latencies)

results = []
for n in SIZES:
    docs = make_documents(n)
    build, search = benchmark(docs)
    results.append([n, build * 1000, search])
    print(n, "docs | Indexing:", round(build*1000, 3),
          "ms | Search:", round(search, 3), "ms")

df = pd.DataFrame(results,
    columns=["Documents","Indexing_Time_ms","Average_Search_Latency_ms"])
df.to_csv("faiss_results.csv", index=False)

plt.figure()
plt.plot(df["Documents"], df["Indexing_Time_ms"], marker="o")
plt.xlabel("Number of Documents")
plt.ylabel("Indexing Time (ms)")
plt.title("FAISS Indexing Time")
plt.grid(True)
plt.savefig("faiss_indexing_time.png", dpi=200)
plt.show()

plt.figure()
plt.plot(df["Documents"], df["Average_Search_Latency_ms"], marker="o")
plt.xlabel("Number of Documents")
plt.ylabel("Average Search Latency (ms)")
plt.title("FAISS Search Latency")
plt.grid(True)
plt.savefig("faiss_search_latency.png", dpi=200)
plt.show()

print(df)
```

IDLE: Open IDLE → File → New File → paste the code → Save as faiss_experiment.py → press F5.

Results and Scalability Analysis

Documents	Indexing Time (ms)	Average Search Latency (ms)
100	Measured during execution	Measured during execution
500	Measured during execution	Measured during execution
1,000	Measured during execution	Measured during execution
5,000	Measured during execution	Measured during execution
10,000	Measured during execution	Measured during execution

Graphs

The program creates faiss_indexing_time.png and faiss_search_latency.png. The X-axis represents the number of documents. The Y-axis represents the corresponding measured time.

Analysis

- Indexing time generally increases as more vectors are inserted.
- IndexFlatIP performs exact comparisons, so search work increases with the number of indexed vectors.

- Small timing fluctuations are expected because CPU load and operating-system scheduling affect very short measurements.
- The 10,000-document experiment makes the scaling trend easier to observe.
- For much larger collections, IVF or HNSW can be considered when exact flat search becomes costly.

Conclusion

FAISS provides efficient vector similarity search. The experiment measures how index construction and exact-search latency change as the collection grows. The CSV file and graphs provide the actual experimental evidence.

QUESTION 18

Aim

To develop a persistent ChromaDB semantic-search application, store documents and embeddings, terminate the application, restart it, reopen the same collection, and verify that retrieval remains available.

Objectives

- Create a persistent ChromaDB database.
- Generate document embeddings.
- Insert IDs, documents and embeddings.
- Perform a query before termination.
- Restart the application.
- Reopen the existing collection.
- Perform retrieval after restart.
- Explain the importance of persistence.

Method

Sentence Transformers converts documents into dense numerical embeddings. ChromaDB stores the documents and embeddings in a persistent database directory. A second Python process uses the same directory and collection name to verify that the stored data remains available.

Workflow

Documents → Sentence Transformer → Embeddings → Persistent ChromaDB Collection → Disk → Restart → Reopen Collection → Semantic Search

Documents

- Machine learning is used to build predictive models from data.
- Natural language processing helps computers understand human language.
- Deep learning uses neural networks with multiple computational layers.
- Vector databases store embeddings for similarity search.
- Information retrieval finds relevant documents for a user query.
- FAISS is a library for efficient similarity search over vectors.
- ChromaDB provides collections for storing documents and embeddings.
- Semantic search retrieves content based on meaning rather than exact words.

First Run – Insert Documents and Search

```
import chromadb
from sentence_transformers import SentenceTransformer

DB_PATH = "./chroma_persistent_db"
COLLECTION_NAME = "semantic_documents"

documents = [
    "Machine learning is used to build predictive models from data.",
    "Natural language processing helps computers understand human language.",
    "Deep learning uses neural networks with multiple computational layers.",
    "Vector databases store embeddings for similarity search.",
    "Information retrieval finds relevant documents for a user query.",
    "FAISS is a library for efficient similarity search over vectors.",
    "ChromaDB provides collections for storing documents and embeddings.",
    "Semantic search retrieves content based on meaning rather than exact words."
]

ids = ["doc_" + str(i) for i in range(len(documents))]

model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(
    documents, normalize_embeddings=True
).tolist()

client = chromadb.PersistentClient(path=DB_PATH)
collection = client.get_or_create_collection(
    name=COLLECTION_NAME
)

collection.upsert(
    ids=ids,
    documents=documents,
    embeddings=embeddings
)

print("Documents inserted.")
print("Collection count:", collection.count())

query = "How are text meanings represented for similarity search?"
query_embedding = model.encode(
    [query], normalize_embeddings=True
).tolist()

result = collection.query(
    query_embeddings=query_embedding, n_results=3
)
```

```
print("Search before restart:")
for item in result["documents"][0]:
    print(item)

print("Now terminate the program.")
```

First-Run Procedure

1. Open Python IDLE and create a new file.
2. Paste the program and save it as chroma_first.py.
3. Press F5 to run.
4. Confirm that the collection count is 8.
5. Observe the returned documents.
6. Terminate the application.
7. Do not delete the chroma_persistent_db folder.

Restart – Reopen the Persistent Collection

```
import chromadb
from sentence_transformers import SentenceTransformer

DB_PATH = "./chroma_persistent_db"
COLLECTION_NAME = "semantic_documents"

model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.PersistentClient(path=DB_PATH)

collection = client.get_collection(
    name=COLLECTION_NAME
)

print("Collection reopened successfully.")
print("Documents after restart:", collection.count())

query = "Which technology stores embeddings for semantic similarity?"
query_embedding = model.encode(
    [query], normalize_embeddings=True
).tolist()

result = collection.query(
    query_embeddings=query_embedding, n_results=3
)

print("Search after restart:")
for item in result["documents"][0]:
    print(item)
```

Restart Procedure

1. Terminate the first program.

2. Open a new IDLE file.

3. Paste the restart program and save it as chroma_restart.py.

4. Press F5.

5. The same persistent path is opened.

6. Check that the collection count is 8.

7. Observe the retrieved documents.

Stage	Expected Result
Initial insertion	8 documents stored
Search before restart	Relevant documents returned
Application termination	Database folder remains
Restart	Existing collection opens
Count after restart	8 documents
Search after restart	Relevant documents returned

Analysis, Importance of Persistence and Conclusion

Why Persistence Is Important

- Stored data survives application termination.
- Embeddings do not need to be regenerated after every restart.
- Startup can be faster because an existing vector collection is reused.
- A semantic-search system can maintain a long-lived knowledge base.
- Persistence is useful for document-search and retrieval-augmented generation applications.
- Application lifecycle is separated from stored vector-data lifecycle.

Analysis

The first program creates a PersistentClient and stores the collection under a specified database path. Documents and embeddings are inserted and a semantic query is performed. When the application terminates, the database directory remains on disk.

The second program creates a new PersistentClient using the same path and obtains the existing collection. The collection count and successful semantic query demonstrate that the data survived the restart.

Expected Result

After the first run, the collection should contain eight documents. After restarting, the same collection should open and report the stored count. A new semantic query should return relevant documents.

Execution Note

Install the packages before running the programs in IDLE. The Sentence Transformer model may download the first time it is used. ChromaDB APIs can vary by version; the required persistence test remains the same: create persistent collection, store embeddings, terminate, reopen, and retrieve.

Overall Conclusion

The FAISS experiment demonstrates vector-search scalability across five dataset sizes. The ChromaDB experiment demonstrates durable storage of documents and embeddings across application restarts. Both experiments provide the required implementation, measurement, retrieval and analysis.

Output Files

- faiss_results.csv
- faiss_indexing_time.png
- faiss_search_latency.png
- chroma_persistent_db/